

QuantumTransportEOM

Detailed User Manual

Project Documentation

March 4, 2026

Contents

1	Purpose and Scope	2
2	Project Layout	2
3	Installation and Environment	2
3.1	Python Requirements	2
3.2	Editable Install	3
3.3	Running the Demo	3
3.4	Running Tests	3
3.5	Automation Script	3
4	Mathematical Conventions	3
4.1	Basis and Indexing	3
4.2	Rashba-Hubbard Ring Hamiltonian	3
4.3	Green Functions	4
4.4	Transport Quantities	4
5	Module-by-Module API Reference	4
5.1	<code>quantum_transport.algebra</code>	4
5.2	<code>quantum_transport.eom</code>	5
5.3	<code>quantum_transport.hamiltonians</code>	5
5.4	<code>quantum_transport.greens</code>	5
5.5	<code>quantum_transport.mean_field</code>	5
5.6	<code>quantum_transport.transport</code>	6
6	End-to-End Workflows	6
6.1	Workflow A: HF + Current + Drude	6
6.2	Workflow B: Symbolic EOM Closure	7

6.3 Workflow C: Real to Reciprocal Space	7
7 Using run_all.ps1	8
8 Testing and Validation	8
9 Troubleshooting	8
9.1 Import Errors	8
9.2 pytest Missing	9
9.3 No PDF Generated	9
9.4 HF Not Converging	9
10 Practical Notes for Research Use	9
11 Extension Roadmap	10
12 License and Citation	10
A Quick API Summary	10

1 Purpose and Scope

QuantumTransportEOM is a standalone Python package for analytical and numerical workflows in quantum transport theory, with special focus on:

- commutator and anticommutator calculations for operator algebras,
- equation-of-motion (EOM) closure analysis,
- Rashba-Hubbard tight-binding ring Hamiltonians in real space and reciprocal space,
- retarded, advanced, lesser, and greater Green functions in equilibrium,
- self-consistent Hartree-Fock (HF) loops,
- transport observables (current density, persistent current, and Drude finite-difference proxy).

This manual describes the full package architecture, installation, API behavior, formula conventions, and practical workflows.

2 Project Layout

The standalone folder is:

```
QuantumTransportEOM/  
  pyproject.toml  
  setup.py  
  run_all.ps1  
  README.md  
  src/quantum_transport/  
    __init__.py  
    algebra.py  
    eom.py  
    hamiltonians.py  
    greens.py  
    mean_field.py  
    transport.py  
  examples/  
    demo.py  
  tests/  
    test_quantum_transport.py  
  docs/  
    user_manual.tex  
    user_manual.pdf
```

3 Installation and Environment

3.1 Python Requirements

- Python 3.9 or newer.
- Package dependencies:
 - numpy
 - sympy

3.2 Editable Install

From QuantumTransportEOM/:

```
python -m pip install -e .
```

3.3 Running the Demo

```
$env:PYTHONPATH=".\"src"
python .\examples\demo.py
```

3.4 Running Tests

```
python -m pip install pytest
python -m pytest .\tests\test_quantum_transport.py
```

3.5 Automation Script

Use the provided script:

```
.\run_all.ps1
```

Useful switches:

- -SkipInstall
- -SkipDemo
- -SkipTests
- -InstallPytest
- -BuildManual

4 Mathematical Conventions

4.1 Basis and Indexing

In the numerical modules, the spinful site basis is ordered as

$$(0, \uparrow), (0, \downarrow), (1, \uparrow), (1, \downarrow), \dots, (N-1, \uparrow), (N-1, \downarrow).$$

Hence the Hamiltonian dimension is $2N \times 2N$.

4.2 Rashba-Hubbard Ring Hamiltonian

The implemented mean-field real-space Hamiltonian is:

$$H = \sum_{n,\sigma} E'_{n\sigma} c_{n\sigma}^\dagger c_{n\sigma} + \sum_n \left(c_{n+1}^\dagger \Gamma_{n+1,n} c_n + \text{h.c.} \right), \quad (1)$$

$$E'_{n\uparrow} = E_{n\uparrow} + U \langle n_{n\downarrow} \rangle, \quad E'_{n\downarrow} = E_{n\downarrow} + U \langle n_{n\uparrow} \rangle. \quad (2)$$

The nearest-neighbor spin matrix $\Gamma_{n+1,n}$ contains both hopping and Rashba terms with Peierls phase

$$\theta = \frac{2\pi}{N} \frac{\phi}{\phi_0}.$$

4.3 Green Functions

For a quadratic Hamiltonian H :

$$G^r(\omega) = [(\omega + i\eta)I - H]^{-1}, \quad (3)$$

$$G^a(\omega) = [(\omega - i\eta)I - H]^{-1}. \quad (4)$$

Equilibrium lesser and greater functions:

$$G^<(\omega) = f(\omega) (G^a(\omega) - G^r(\omega)), \quad (5)$$

$$G^>(\omega) = (f(\omega) - 1) (G^a(\omega) - G^r(\omega)), \quad (6)$$

with Fermi function $f(\omega)$.

4.4 Transport Quantities

The package computes:

- $J_c(\omega)$: current density integrand built from matrix elements of $G^r - G^a$, hopping γ , Rashba λ_R , and flux phase factors.
- $I_c(\phi)$: persistent current by numerical integration over ω :

$$I_c(\phi) = \int J_c(\omega) d\omega.$$

- Drude proxy:

$$D \sim \frac{dI}{d\phi} \approx \frac{I(\phi + \delta\phi) - I(\phi - \delta\phi)}{2\delta\phi}.$$

5 Module-by-Module API Reference

5.1 `quantum_transport.algebra`

Purpose: symbolic noncommutative algebra utilities.

Function	Behavior
<code>commutator(a,b,simplify=True)</code>	Returns $[a,b] = ab - ba$. Uses SymPy expand and optional simplify.
<code>anticommutator(a,b,simplify=True)</code>	Returns $\{a,b\} = ab + ba$.
<code>decompose_in_basis(expr, basis)</code>	Returns (c,r) such that $expr = \sum_j c_j basis_j + r$. Useful for EOM projection diagnostics.
<code>operator_symbols(prefix, size)</code>	Creates noncommutative SymPy symbols (for example <code>c0, c1, ...</code>).

5.2 quantum_transport.eom

Purpose: EOM closure checks and symbolic retarded Green solver on closed bases.

Object / Function	Behavior
EOMClosureResult	Dataclass with fields: <code>operators</code> , <code>eom_matrix</code> , <code>residuals</code> . Property <code>is_closed</code> is true if all residuals are zero.
<code>check_eom_closure(operators, eom_matrix, residuals)</code>	Builds matrix M from $[O_i, H] = \sum_j M_{ij} O_j + \text{residual}_i$.
<code>retarded_green_from_eom(Solver(C+wieta))</code>	Solves $(C + i\eta)I - M)G^r = C$. Raises if basis is not closed.
<code>source_anticommutator_matrix(A,B)</code>	Builds symbolic $C_{ij} = \{A_i, B_j\}$.

5.3 quantum_transport.hamiltonians

Purpose: numerical Hamiltonian builders and real $\leftrightarrow$ k-space transforms.

Function	Behavior
<code>build_rashba_hubbard_ring(n_sites, spin_dim=2)</code>	Builds 2D $N \times N$ Hermitian matrix with on-site MF shift, NN hopping, Rashba coupling, and Peierls phase.
<code>real_to_k_space(h_real, n_sites, spin_dim=2)</code>	Applies discrete Fourier transform over site index and returns (H_k, k_values) .
<code>split_k_blocks(h_k, n_sites, spin_dim=2)</code>	Splits H_k into contiguous $spin_dim \times spin_dim$ diagonal blocks (per k index in the transformed ordering).

5.4 quantum_transport.greens

Purpose: retarded/advanced and equilibrium Keldysh components.

Function	Behavior
<code>fermi_dirac(ω, mu=0, T=0)</code>	At $T = 0$: returns 1 below μ , 0 above, and 0.5 at equality. At $T > 0$: returns logistic form.
<code>retarded_green(h, ω, eta)</code>	Matrix inverse for scalar or vector ω . Returns shape (dim, dim) or (n_ω, dim, dim) .
<code>advanced_green(h, ω, eta)</code>	Same shape behavior as retarded, with $-i\eta$.
<code>spectral_function(G_r, G_a)</code>	Returns $A = i(G^r - G^a)$.
<code>lesser_green_equilibrium(ω)</code>	Returns $G^< = f(G^a - G^r)$. Supports scalar/vector ω .
<code>greater_green_equilibrium(ω)</code>	Returns $G^> = (f - 1)(G^a - G^r)$.

5.5 quantum_transport.mean_field

Purpose: self-consistent Hartree-Fock loop for on-site Hubbard decoupling.

Object / Function	Behavior
HartreeFockResult	Dataclass containing final Hamiltonian, eigenpairs, $n_\uparrow$, $n_\downarrow$, occupations, iteration count, and convergence flag.
<code>hartree_fock_self_consistent(ω)</code>	Iterative density update with linear mixing until max density change $< tol$ or <code>max_iter</code> .

Important implementation details:

- Zero-temperature filling with optional fractional last occupation for non-integer N_e .
- Density matrix construction:

$$\rho = V \text{diag}(\text{occ}) V^\dagger.$$

- Spin densities sampled from diagonal entries in interleaved basis:

$$n_\uparrow(n) = \rho_{2n,2n}, \quad n_\downarrow(n) = \rho_{2n+1,2n+1}.$$

5.6 quantum_transport.transport

Purpose: transport observables from Green functions.

Function	Behavior
<code>current_density_omega(...)</code>	Computes $J_c(\omega)$ in the manuscript-style structure, combining spin-preserving and Rashba spin-flip terms. Returns real scalar.
<code>persistent_current(...)</code>	Builds G^r, G^a over ω grid and integrates $J_c(\omega)$ with trapezoidal rule.
<code>drude_weight(Iplus, Iminus, alpha)</code>	Finite difference $(I_+ - I_-)/(2d\phi)$.

6 End-to-End Workflows

6.1 Workflow A: HF + Current + Drude

```
import numpy as np
from quantum_transport import (
    hartree_fock_self_consistent,
    persistent_current,
    drude_weight,
)

n_sites = 8
n_electrons = 8
gamma = 1.0
lambda_r = 0.4
u_hubbard = 0.4
phi = 0.2
dphi = 1e-3

hf = hartree_fock_self_consistent(
    n_sites=n_sites,
    n_electrons=n_electrons,
    gamma=gamma,
    lambda_r=lambda_r,
    phi_over_phi0=phi,
    u_hubbard=u_hubbard,
)

mu = 0.5 * (hf.eigenvalues[n_electrons - 1] + hf.eigenvalues[
    n_electrons])
omega_grid = np.linspace(-6.0, 6.0, 2001)
```

```

I0 = persistent_current(
    hamiltonian=hf.hamiltonian,
    omega_grid=omega_grid,
    n_sites=n_sites,
    gamma=gamma,
    lambda_r=lambda_r,
    phi_over_phi0=phi,
    mu=mu,
)

hf_plus = hartree_fock_self_consistent(n_sites, n_electrons, gamma,
    lambda_r, phi + dphi, u_hubbard)
hf_minus = hartree_fock_self_consistent(n_sites, n_electrons, gamma,
    lambda_r, phi - dphi, u_hubbard)

Iplus = persistent_current(hf_plus.hamiltonian, omega_grid, n_sites,
    gamma, lambda_r, phi + dphi, mu)
Iminus = persistent_current(hf_minus.hamiltonian, omega_grid, n_sites,
    gamma, lambda_r, phi - dphi, mu)

D = drude_weight(Iplus, Iminus, dphi)
print(I0, D)

```

6.2 Workflow B: Symbolic EOM Closure

```

import sympy as sp
from quantum_transport import commutator, anticommutator,
    check_eom_closure

c0, c1 = sp.symbols("c0 c1", commutative=False)
H = c0*c1 + c1*c0

print(commutator(c0, H))
print(anticommutator(c0, c1))

closure = check_eom_closure([c0, c1], H)
print("Closed basis?", closure.is_closed)
print("Residuals:", closure.residuals)

```

6.3 Workflow C: Real to Reciprocal Space

```

import numpy as np
from quantum_transport import (
    build_rashba_hubbard_ring_real_space,
    real_to_k_space,
    split_k_blocks
)

H = build_rashba_hubbard_ring_real_space(
    n_sites=8,
    gamma=1.0,
    lambda_r=0.3,
    phi_over_phi0=0.1,
)

```



```
Hk, k = real_to_k_space(H, n_sites=8, spin_dim=2)
blocks = split_k_blocks(Hk, n_sites=8, spin_dim=2)

print(k[:3])
print(blocks[0])
```

7 Using run_all.ps1

Basic use:

```
.\run_all.ps1
```

Install package and pytest if missing, run demo and tests, and build manual PDF:

```
.\run_all.ps1 -InstallPytest -BuildManual
```

Run only docs build:

```
.\run_all.ps1 -SkipInstall -SkipDemo -SkipTests -BuildManual
```

8 Testing and Validation

Current tests verify:

- commutator and anticommutator identities,
- Hermiticity of the built ring Hamiltonian,
- spectrum invariance under real-to-k transformation,
- shape consistency of $G^r, G^a, G^<, G^>$,
- execution of HF loop with physically valid returned densities.

Recommended additional tests for research-grade studies:

- convergence sensitivity vs mixing and tolerance,
- finite-size scaling of persistent current with N ,
- cross-check between derivative-of-energy and Green-function current,
- benchmark against exact diagonalization for small N ,
- regression tests for particle-hole symmetry in even bipartite rings.

9 Troubleshooting

9.1 Import Errors

If Python cannot import `quantum_transport`, run:

```
python -m pip install -e .
```

or set:

```
$env:PYTHONPATH=".\"src"
```

9.2 pytest Missing

```
python -m pip install pytest
```

or:

```
.\run_all.ps1 -InstallPytest
```

9.3 No PDF Generated

If `pdflatex` is not installed, install MiKTeX or TeX Live, then run:

```
.\run_all.ps1 -BuildManual -SkipInstall -SkipDemo -SkipTests
```

9.4 HF Not Converging

Try:

- lower `mixing` (for example 0.2),
- higher `max_iter`,
- relaxed `tol` for exploratory scans,
- better initial densities if a specific ordered phase is expected.

10 Practical Notes for Research Use

- The equilibrium lesser/greater relations implemented here assume closed-system equilibrium.
- For open systems with leads, add self-energies and Keldysh equations with contact distribution functions.
- The current implementation is optimized for clarity and correctness, not large-scale performance.
- For production sweeps, vectorize parameter scans externally and cache Hamiltonians where possible.

11 Extension Roadmap

Suggested next extensions:

1. Add lead self-energy support for nonequilibrium transport.
2. Add finite-temperature HF occupations in the self-consistent loop.
3. Add exact diagonalization backend for small clusters.
4. Add automatic symmetry checks (particle-hole, time reversal, inversion where applicable).
5. Add plotting utilities for spectra, $I(\phi)$, and $D(\lambda_R)$.

12 License and Citation

If used in academic work, cite the corresponding project repository and include the exact commit hash used to generate numerical results.

A Quick API Summary

Name	Summary
<code>commutator, anticommutator</code>	Symbolic operator algebra primitives.
<code>check_eom_closure</code>	EOM matrix and residual closure check.
<code>build_rashba_hubbard_ring_real_space</code>	Core spinful real-space Hamiltonian constructor.
<code>real_to_k_space,</code> <code>split_k_blocks</code>	Reciprocal-space transform and block extraction.
<code>retarded_green,</code> <code>advanced_green</code>	Frequency-domain matrix Green functions.
<code>lesser_green_equilibrium,</code> <code>greater_green_equilibrium</code>	Equilibrium Keldysh components.
<code>hartree_fock_self_consistent</code>	Mean-field self-consistent solver.
<code>current_density_omega,</code> <code>persistent_current</code>	Transport observables from Green functions.
<code>drude_weight</code>	Finite difference Drude proxy from current response.